

Biophysical Inference from Particle Trajectories Using Neural Posterior Estimation

Alireza Yazdani, Tina Torabi, Fatemeh Saghaififar

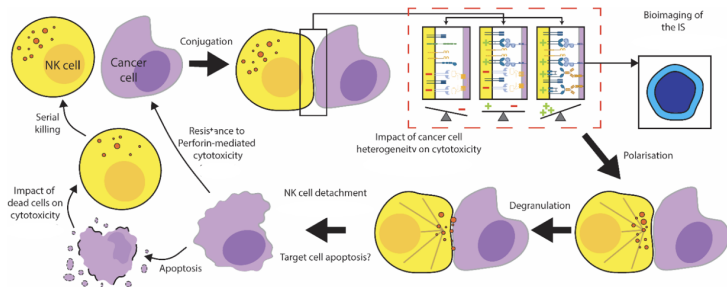
Math 604E final project



- 1 Motivation
- 2 Methodology
- 3 Results
- 4 Future Plans
- 5 Reference

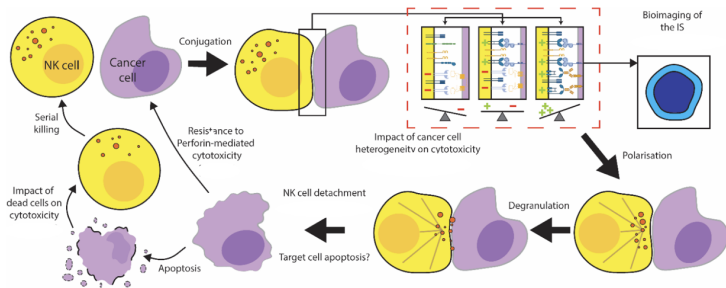
Natural Killer (NK) Cells

- Natural killer (NK) cells are cytotoxic lymphocytes of the **innate** immune system, providing rapid first-line defense.
- Simple and direct activation of these cells makes them important in cancer immunity.



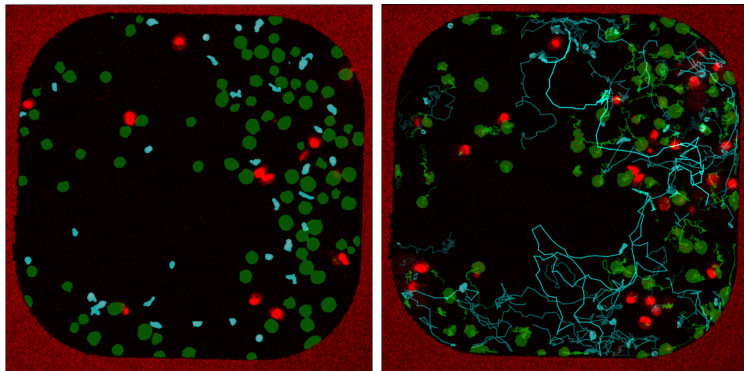
Natural Killer (NK) Cells

- Natural killer (NK) cells are cytotoxic lymphocytes of the **innate** immune system, providing rapid first-line defense.
- Simple and direct activation of these cells makes them important in cancer immunity.



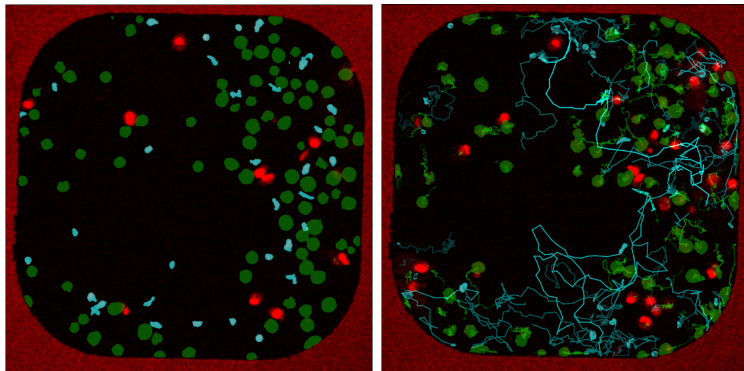
How much of the story lives in the trajectory data of these cells?

Observations



Green: Live cancer cells, Blue: NK cells, Red: Dead cancer cells

Observations



Green: Live cancer cells, Blue: NK cells, Red: Dead cancer cells

Core question

How far can we get from trajectories alone? What mechanisms of NK motility and killing are actually encoded in the tracks?

Cell Path as Random Walk

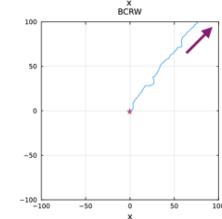
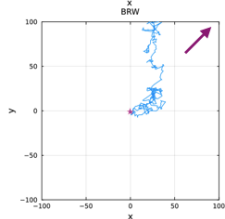
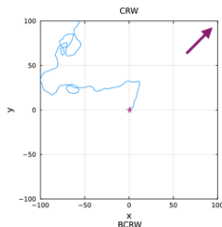
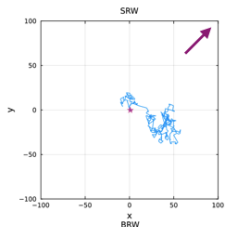
In Random Walk (RW) models, new direction at each time depends on:

- **Bias** directional preference due to external cues
- **Correlation** Internal tendency to continue in the current direction

Cell Path as Random Walk

In Random Walk (RW) models, new direction at each time depends on:

- **Bias** directional preference due to external cues
- **Correlation** Internal tendency to continue in the current direction



SRW: Diffusion process

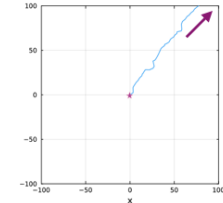
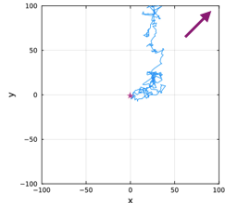
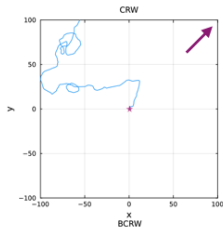
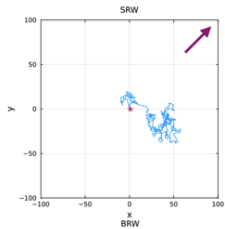
BRW: Drift-Diffusion process

$$\text{probability density} = p(\vec{X}(t), t)$$

Cell Path as Random Walk

In Random Walk (RW) models, new direction at each time depends on:

- **Bias** directional preference due to external cues
- **Correlation** Internal tendency to continue in the current direction



SRW: Diffusion process

BRW: Drift-Diffusion process

$$\text{probability density} = p(\vec{X}(t), t)$$

CRW: Persistent Diffusion process

BCRW: Persistent Drift-Diffusion process

$$\text{probability density} = p(\vec{X}(t), \vec{V}(t), t)$$

- In absence of correlation, p has closed form:

$$p(\vec{X}, t) = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{(x - vt)^2}{4Dt}\right)$$

- With directional correlation, p admits no closed-form solution. Assuming constant speed, the velocity changes only through reorientation.

We model reorientation via a **memory kernel** $K(\theta', \theta)$, the probability density of turning from direction θ to θ' . A standard choice is the **von Mises** distribution:

$$K(\theta', \theta) = \frac{1}{2\pi I_0(\kappa)} e^{\kappa \cos(\theta - \theta' + \mu)}$$

From Turning Kernel to Biophysical Parameters

$$K(\theta', \theta) = \frac{1}{2\pi I_0(\kappa)} e^{\kappa \cos(\theta - \theta' + \mu)}$$

- Environmental cues can introduce a directional bias, creating a preferred direction of motion. These effects enter the model through the mean term in the kernel above. Focusing on chemotaxis as the external bias, the literature gives the mean in the form:

$$\mu = -d_\tau f(\theta - \theta_0(x)), \quad \theta_0(x) \propto \nabla C(x)$$

- Interpretation of kernel parameters:

κ : correlation - swimming ability

d_τ : bias - sensing strength

- Result: a single kernel captures both persistence (κ) and bias (d_τ) in trajectory dynamics. The goal is to infer these parameters.

From Turning Kernel to Biophysical Parameters

- Assume each target cell is a point source emitting chemoattractant that diffuses isotropically, so $C = C(r)$ depends only on the distance r to the target and satisfies

$$D\nabla^2 C(r) - \lambda C(r) = -Q\delta(r).$$

- The (asymptotic) radial solution is

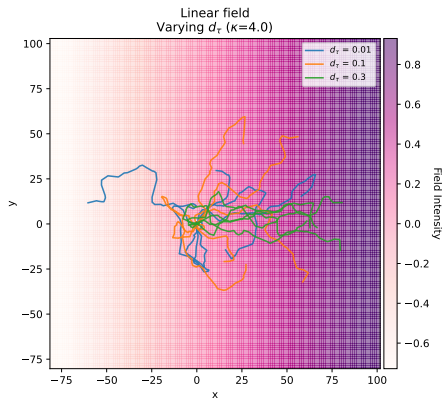
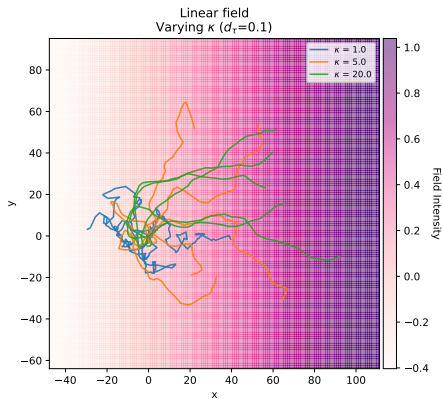
$$C(r) = \frac{Q}{2\pi D} K_0\left(\frac{r}{L}\right), \quad L = \sqrt{\frac{D}{\lambda}},$$

introducing the additional model parameter L .

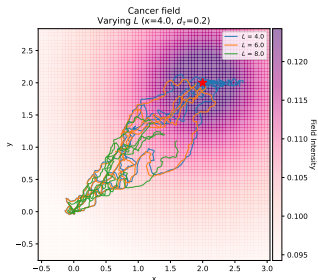
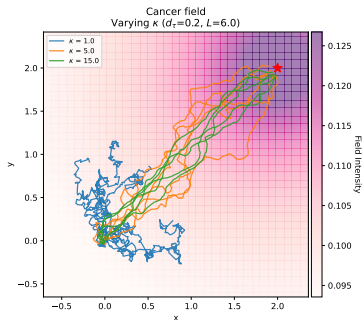
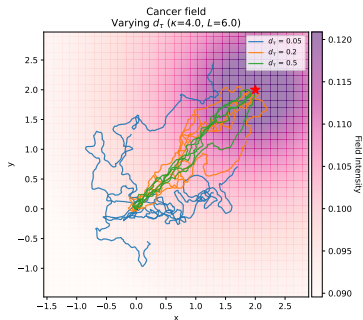
- As a result, for the simple setup with one target cell, the parameter space is

$$\Theta = \{\kappa, d_\tau, L\}.$$

Trajectory Visualization - Fixed θ_0



Trajectory Visualization - Variable $\theta_0(x)$



Model Choice for Reliable Parameter Inference

- The trajectory model is simulator-defined, so we can sample

$$x \sim p(x \mid \theta),$$

but we cannot evaluate $p(x_o \mid \theta)$ analytically.

- With directional correlation, p has no closed form, so inference must use likelihood-free methods working with simulated pairs (θ, x) .
- A useful method is **stochastic variational inference (SVI)**, which learns an explicit surrogate posterior that scales to high-dimensional trajectory data.
- Compared to other likelihood-free methods, SVI avoids manual summary statistics and distance thresholds, uses simulations more efficiently, and yields a smooth posterior approximation (rather than an acceptance/rejection sample cloud).

Model Choice for Reliable Parameter Inference

- The trajectory model is simulator-defined, so we can sample

$$x \sim p(x \mid \theta),$$

but we cannot evaluate $p(x_o \mid \theta)$ analytically.

- With directional correlation, p has no closed form, so inference must use likelihood-free methods working with simulated pairs (θ, x) .
- A useful method is **stochastic variational inference (SVI)**, which learns an explicit surrogate posterior that scales to high-dimensional trajectory data.
- Compared to other likelihood-free methods, SVI avoids manual summary statistics and distance thresholds, uses simulations more efficiently, and yields a smooth posterior approximation (rather than an acceptance/rejection sample cloud).

A suitable algorithm is **Neural Posterior Estimation (NPE)**, which infers parameters directly from simulated data.

What we have.

- An observed trajectory

$$x_o = ((x_1, y_1), \dots, (x_T, y_T)).$$

Neural Posterior Estimation (NPE)

What we have.

- An observed trajectory

$$x_o = ((x_1, y_1), \dots, (x_T, y_T)).$$

- A prior distribution over parameters θ , e.g.

- $\theta = (\kappa, d_T)$
- $\theta = (\kappa, d_T, L)$

What we have.

- An observed trajectory

$$x_o = ((x_1, y_1), \dots, (x_T, y_T)).$$

- A prior distribution over parameters θ , e.g.

- $\theta = (\kappa, d_T)$
- $\theta = (\kappa, d_T, L)$

- A simulator that can generate trajectories

$$x \sim p(x \mid \theta)$$

for any chosen θ .

Neural Posterior Estimation (NPE)

What we have.

- An observed trajectory

$$x_o = ((x_1, y_1), \dots, (x_T, y_T)).$$

- A prior distribution over parameters θ , e.g.

- $\theta = (\kappa, d_T)$
- $\theta = (\kappa, d_T, L)$

- A simulator that can generate trajectories

$$x \sim p(x \mid \theta)$$

for any chosen θ .

What we want.

The *posterior distribution* over parameters that could have generated the observed trajectory:

$$p(\theta \mid x_o),$$

rather than a single point estimate.

Neural Posterior Estimation (NPE)

What we have.

- An observed trajectory

$$x_o = ((x_1, y_1), \dots, (x_T, y_T)).$$

- A prior distribution over parameters θ , e.g.

- $\theta = (\kappa, d_T)$

- $\theta = (\kappa, d_T, L)$

- A simulator that can generate trajectories

$$x \sim p(x \mid \theta)$$

for any chosen θ .

What we want.

The *posterior distribution* over parameters that could have generated the observed trajectory:

$$p(\theta \mid x_o),$$

rather than a single point estimate.

The obstacle. We don't have access to the likelihood $p(x \mid \theta)$. Without the likelihood, standard Bayesian inference is not possible.

Neural Posterior Estimation (NPE)

Key idea (NPE). Neural Posterior Estimation learns an approximation of the posterior $p(\theta \mid x)$ *directly*, without evaluating the likelihood.

Neural Posterior Estimation (NPE)

Key idea (NPE). Neural Posterior Estimation learns an approximation of the posterior $p(\theta \mid x)$ *directly*, without evaluating the likelihood.

Step 1: Simulate training data.

$$\theta^{(i)} \sim p(\theta), \quad x^{(i)} \sim p(x \mid \theta^{(i)}).$$

Neural Posterior Estimation (NPE)

Key idea (NPE). Neural Posterior Estimation learns an approximation of the posterior $p(\theta \mid x)$ *directly*, without evaluating the likelihood.

Step 1: Simulate training data.

$$\theta^{(i)} \sim p(\theta), \quad x^{(i)} \sim p(x \mid \theta^{(i)}).$$

Step 2: Train the NPE model. Train a neural density estimator $q_\phi(\theta \mid x)$ by minimizing

$$\mathcal{L}(\phi) = \mathbb{E}_{p(\theta) p(x|\theta)}[-\log q_\phi(\theta \mid x)].$$

Neural Posterior Estimation (NPE)

Key idea (NPE). Neural Posterior Estimation learns an approximation of the posterior $p(\theta | x)$ *directly*, without evaluating the likelihood.

Step 1: Simulate training data.

$$\theta^{(i)} \sim p(\theta), \quad x^{(i)} \sim p(x | \theta^{(i)}).$$

Step 2: Train the NPE model. Train a neural density estimator $q_\phi(\theta | x)$ by minimizing

$$\mathcal{L}(\phi) = \mathbb{E}_{p(\theta) p(x|\theta)}[-\log q_\phi(\theta | x)].$$

Interpretation. This objective minimizes

$$\text{KL}(p(\theta | x) \parallel q_\phi(\theta | x)) \quad \text{on average over } x,$$

so in the infinite-data limit, $q_\phi(\theta | x)$ recovers the true posterior.

Neural Posterior Estimation (NPE)

Key idea (NPE). Neural Posterior Estimation learns an approximation of the posterior $p(\theta | x)$ *directly*, without evaluating the likelihood.

Step 1: Simulate training data.

$$\theta^{(i)} \sim p(\theta), \quad x^{(i)} \sim p(x | \theta^{(i)}).$$

Step 2: Train the NPE model. Train a neural density estimator $q_\phi(\theta | x)$ by minimizing

$$\mathcal{L}(\phi) = \mathbb{E}_{p(\theta) p(x|\theta)}[-\log q_\phi(\theta | x)].$$

Interpretation. This objective minimizes

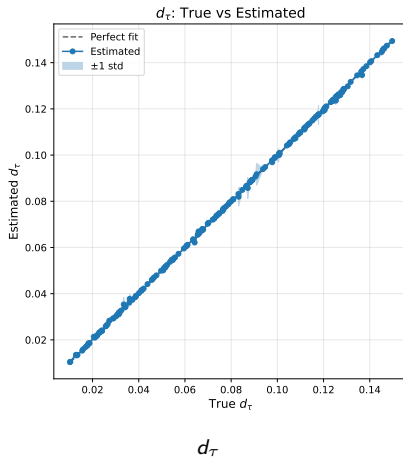
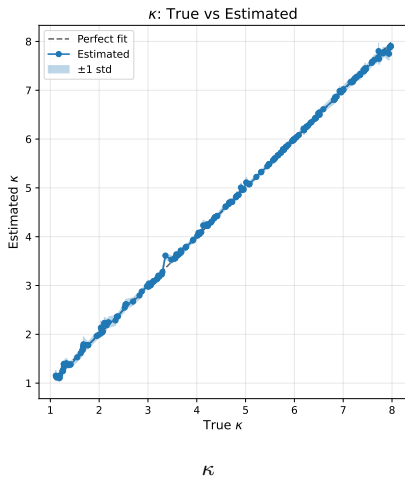
$$\text{KL}(p(\theta | x) \parallel q_\phi(\theta | x)) \quad \text{on average over } x,$$

so in the infinite-data limit, $q_\phi(\theta | x)$ recovers the true posterior.

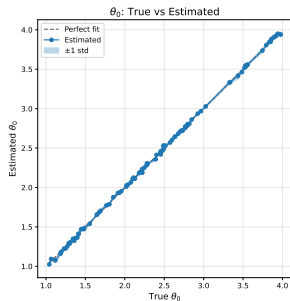
Step 3: Perform inference. Given the observed trajectory x_o ,

$$q_\phi(\theta | x_o) \approx p(\theta | x_o).$$

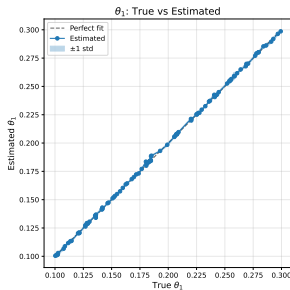
Results NPE (Linear)



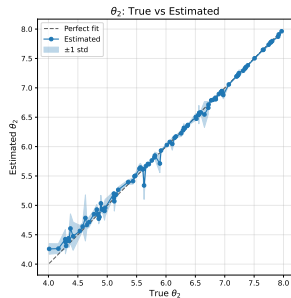
Results: NPE (Cancer)



κ



d_T



L

What NPE does well. It learns the full posterior $p(\theta \mid x)$ using only simulated data.

What NPE does well. It learns the full posterior $p(\theta \mid x)$ using only simulated data.

But think about how the training data is generated. We sample θ from a broad prior and simulate trajectories x .

What NPE does well. It learns the full posterior $p(\theta \mid x)$ using only simulated data.

But think about how the training data is generated. We sample θ from a broad prior and simulate trajectories x .

Most of these trajectories look nothing like the observed one. They correspond to parameter values that are inconsistent with x_o .

What NPE does well. It learns the full posterior $p(\theta \mid x)$ using only simulated data.

But think about how the training data is generated. We sample θ from a broad prior and simulate trajectories x .

Most of these trajectories look nothing like the observed one. They correspond to parameter values that are inconsistent with x_o .

So what's the problem? A large fraction of our simulations end up being uninformative for the posterior we actually care about.

What NPE does well. It learns the full posterior $p(\theta \mid x)$ using only simulated data.

But think about how the training data is generated. We sample θ from a broad prior and simulate trajectories x .

Most of these trajectories look nothing like the observed one. They correspond to parameter values that are inconsistent with x_o .

So what's the problem? A large fraction of our simulations end up being uninformative for the posterior we actually care about.

What should we do now?

A natural question. If we already know roughly what kinds of simulations look like x_o , why keep sampling blindly from the prior?

A Natural Next Step

A natural question. If we already know roughly what kinds of simulations look like x_o , why keep sampling blindly from the prior?

Intuition. Use the information learned so far to guide where we simulate next.

A natural question. If we already know roughly what kinds of simulations look like x_o , why keep sampling blindly from the prior?

Intuition. Use the information learned so far to guide where we simulate next.

Key idea. After an initial round of simulations, we concentrate future simulations in parts of parameter space that are more consistent with the observed data.

A natural question. If we already know roughly what kinds of simulations look like x_o , why keep sampling blindly from the prior?

Intuition. Use the information learned so far to guide where we simulate next.

Key idea. After an initial round of simulations, we concentrate future simulations in parts of parameter space that are more consistent with the observed data.

This leads to: *Sequential Neural Posterior Estimation (SNPE).*

Round 1. We start just like before: sample parameters from the prior, simulate trajectories, and train an initial posterior.

Round 1. We start just like before: sample parameters from the prior, simulate trajectories, and train an initial posterior.

What we learn. Even after just one round, we already have a sense of where in parameter space the trajectories start to look like x_o .

Round 1. We start just like before: sample parameters from the prior, simulate trajectories, and train an initial posterior.

What we learn. Even after just one round, we already have a sense of where in parameter space the trajectories start to look like x_o .

What changes next. Instead of going back to the prior, we sample new parameters from the current posterior which we call the **proposal**.

Round 1. We start just like before: sample parameters from the prior, simulate trajectories, and train an initial posterior.

What we learn. Even after just one round, we already have a sense of where in parameter space the trajectories start to look like x_o .

What changes next. Instead of going back to the prior, we sample new parameters from the current posterior which we call the **proposal**.

Repeat. Each round focuses the simulations a bit more, until the posterior concentrates around the values that best explain the data.

Round 1. We start just like before: sample parameters from the prior, simulate trajectories, and train an initial posterior.

What we learn. Even after just one round, we already have a sense of where in parameter space the trajectories start to look like x_o .

What changes next. Instead of going back to the prior, we sample new parameters from the current posterior which we call the **proposal**.

Repeat. Each round focuses the simulations a bit more, until the posterior concentrates around the values that best explain the data.

Why this matters in practice. This becomes very useful when working with experimental data, where running simulations is expensive and only a small region of parameters ends up explaining what we see.

SNPE Algorithm

Rounds $r = 1, \dots, R$.

- ❶ **Proposal sampling.** Sample parameters from the current proposal:

$$\theta^{(i)} \sim \tilde{p}_r(\theta), \quad \tilde{p}_1(\theta) = p(\theta).$$

- ❷ **Simulation.** Generate synthetic data using the simulator: $x^{(i)} \sim p(x \mid \theta^{(i)})$.

- ❸ **Importance-weighted training.** Train $q_\phi(\theta \mid x)$ by minimizing

$$\mathbb{E} \left[-\log q_\phi(\theta \mid x) \frac{p(\theta)}{\tilde{p}_r(\theta)} \right].$$

Rounds $r = 1, \dots, R$.

- ➊ **Proposal sampling.** Sample parameters from the current proposal:

$$\theta^{(i)} \sim \tilde{p}_r(\theta), \quad \tilde{p}_1(\theta) = p(\theta).$$

- ➋ **Simulation.** Generate synthetic data using the simulator: $x^{(i)} \sim p(x \mid \theta^{(i)})$.

- ➌ **Importance-weighted training.** Train $q_\phi(\theta \mid x)$ by minimizing

$$\mathbb{E} \left[-\log q_\phi(\theta \mid x) \frac{p(\theta)}{\tilde{p}_r(\theta)} \right].$$

Why the reweighting? Since $q_\phi(\theta \mid x)$ is trained on samples

$$\theta \sim \tilde{p}(\theta), \quad x \sim p(x \mid \theta),$$

then it converges to the *proposal posterior*

$$\tilde{p}(\theta \mid x) \propto p(x \mid \theta) \tilde{p}(\theta),$$

rather than the true posterior defined using $p(\theta)$.

Consequence. Training without correction biases the learned posterior toward the proposal.

Rounds $r = 1, \dots, R$.

- ❶ **Proposal sampling.** Sample parameters from the current proposal:

$$\theta^{(i)} \sim \tilde{p}_r(\theta), \quad \tilde{p}_1(\theta) = p(\theta).$$

- ❷ **Simulation.** Generate synthetic data using the simulator: $x^{(i)} \sim p(x \mid \theta^{(i)})$.

- ❸ **Importance-weighted training.** Train $q_\phi(\theta \mid x)$ by minimizing

$$\mathbb{E} \left[-\log q_\phi(\theta \mid x) \frac{p(\theta)}{\tilde{p}_r(\theta)} \right].$$

Why the reweighting? Since $q_\phi(\theta \mid x)$ is trained on samples

$$\theta \sim \tilde{p}(\theta), \quad x \sim p(x \mid \theta),$$

then it converges to the *proposal posterior*

$$\tilde{p}(\theta \mid x) \propto p(x \mid \theta) \tilde{p}(\theta),$$

rather than the true posterior defined using $p(\theta)$.

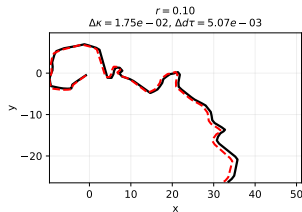
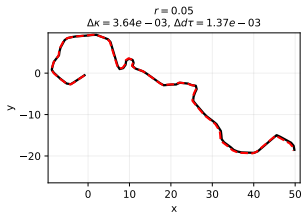
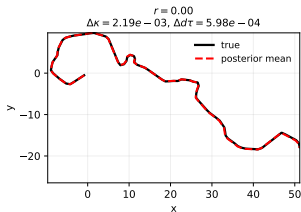
Consequence. Training without correction biases the learned posterior toward the proposal.

Proposal update. We then set the next proposal to the posterior conditioned on the observation:

$$\tilde{p}_{r+1}(\theta) = q_\phi(\theta \mid x_o).$$

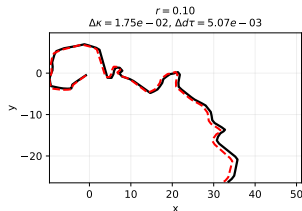
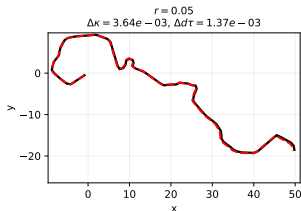
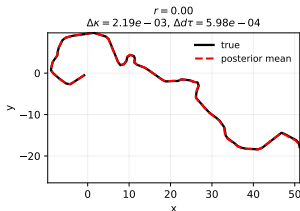
Results SNPE(Linear)

- We start from a set of true parameters that generate a reference trajectory, and train an SNPE posterior on that trajectory.



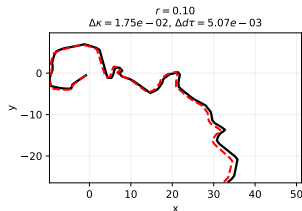
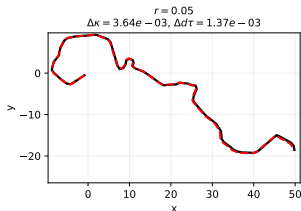
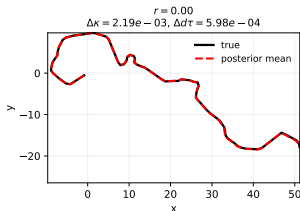
Results SNPE(Linear)

- We start from a set of true parameters that generate a reference trajectory, and train an SNPE posterior on that trajectory.
- **Baseline ($r = 0$):** When we infer parameters from this trajectory and run the simulator again, we recover the trajectory almost exactly.



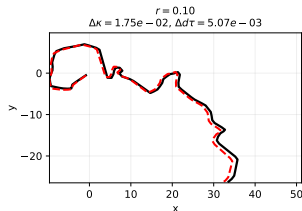
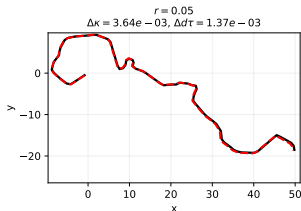
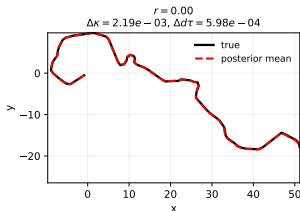
Results SNPE(Linear)

- We start from a set of true parameters that generate a reference trajectory, and train an SNPE posterior on that trajectory.
- **Baseline ($r = 0$):** When we infer parameters from this trajectory and run the simulator again, we recover the trajectory almost exactly.
- Next, we slightly perturb the true parameters, simulate new trajectories, and pass them through the *same* learned posterior.



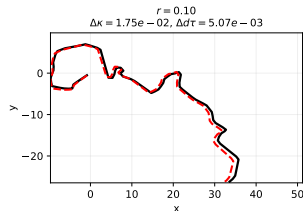
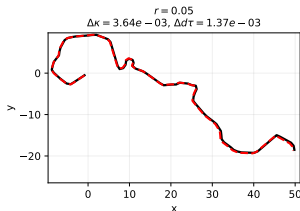
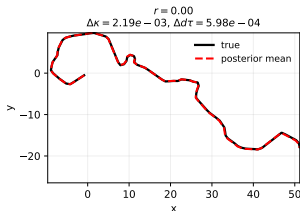
Results SNPE(Linear)

- We start from a set of true parameters that generate a reference trajectory, and train an SNPE posterior on that trajectory.
- **Baseline ($r = 0$):** When we infer parameters from this trajectory and run the simulator again, we recover the trajectory almost exactly.
- Next, we slightly perturb the true parameters, simulate new trajectories, and pass them through the *same* learned posterior.
- For small perturbations, the posterior still recovers the parameters and trajectories very well.

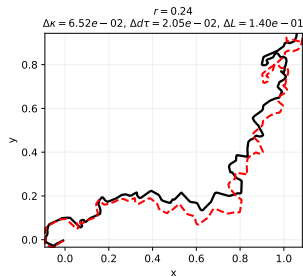
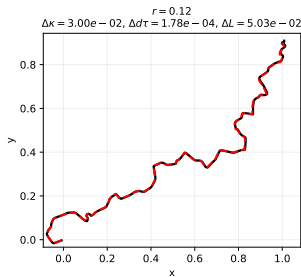
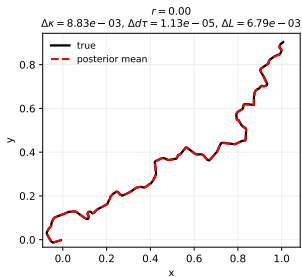


Results SNPE(Linear)

- We start from a set of true parameters that generate a reference trajectory, and train an SNPE posterior on that trajectory.
- **Baseline ($r = 0$):** When we infer parameters from this trajectory and run the simulator again, we recover the trajectory almost exactly.
- Next, we slightly perturb the true parameters, simulate new trajectories, and pass them through the *same* learned posterior.
- For small perturbations, the posterior still recovers the parameters and trajectories very well.
- As the perturbations grow, the recovered trajectories start to drift and we naturally lose accuracy in inferred parameters, which is expected!



Results SNPE (Cancer)



- **More target points:** Extend the model to multiple chemoattractant sources and infer parameters in multi-target environments.
- **Crowding effect:** Include physical crowding and obstacles to model constrained motion in dense cell environments.
- **Experimental validation:** Apply the inference pipeline to real NK-cell tracking data and compare inferred parameters across conditions.
- **Serial killing:** Model repeated kill–detach–reengage dynamics and test how serial killing signatures appear in trajectories.



G. Papamakarios and I. Murray. Fast ϵ free inference of simulation models with Bayesian conditional density estimation. Advances in Neural Information Processing Systems, 2016.



S. Griesemer, D. Cao, Z. Cui, C. Osorio, and Y. Liu. Active Sequential Neural Posterior Estimation for sample efficient simulation based inference. Advances in Neural Information Processing Systems, 2024.



E. A. Codling. Biased random walks in biology. PhD thesis, University of Leeds, 2003.



Griesemer, S., Cao, D., Cui, Z., Osorio, C. and Liu, Y., 2024. Active sequential posterior estimation for sample-efficient simulation-based inference. Advances in Neural Information Processing Systems, 37, pp.127907-127936.



Hoffman, M.D., Blei, D.M., Wang, C. and Paisley, J., 2013. Stochastic variational inference. the Journal of machine Learning research, 14(1), pp.1303-1347.